CS336 Assignment 1 (basics): Writeup

Jacques Attinger

Spring 2026

2 Byte-Pair Encoding (BPE) Tokenizer

2.1 The Unicode Standard

Problem `unicode1`: Understanding Unicode (1 point)

(a) What Unicode character does `chr(0)` return?

Todo

(b) How does this character's string representation (`__repr__()`) differ from its printed representation?

Todo

(c) What happens when this character occurs in text? Check your guess in the Python interpreter.

Todo

2.2 Unicode Encodings

Problem `unicode2`: Unicode Encodings (3 points)

(a) What are some reasons to prefer training our tokenizer on UTF-8 encoded bytes, rather than UTF-16 or UTF-32?

Todo

(b) Consider the (incorrect) function `decode_utf8_bytes_to_str_wrong`. Why is it incorrect? Give an example input byte string that yields incorrect results.

Todo

(c) Give a two-byte sequence that does not decode to any Unicode character(s).

Todo

2.4 BPE Tokenizer Training

Problem `train_bpe`: BPE Tokenizer Training (15 points)

Code deliverable only. No writeup answer.

Problem `train_bpe_tinystories`: BPE Training on TinyStories (2 points)

(a) Train a byte-level BPE tokenizer on TinyStories with a maximum vocabulary size of 10,000, including the `<|endoftext|>` special token. Serialize the vocabulary and merges. How much time and memory did training take? What is the longest token in the vocabulary? Does it make sense?

Todo

(b) Profile your code. What part of the tokenizer training process takes the most time?

Todo

Problem `train_bpe_expts_owt`: BPE Training on OpenWebText (2 points)

(a) Train a byte-level BPE tokenizer on OpenWebText with a maximum vocabulary size of 32,000. Serialize the vocabulary and merges. What is the longest token in the vocabulary? Does it make sense?

Todo

(b) Compare and contrast the tokenizer you get training on TinyStories versus OpenWebText.

Todo

2.6 BPE Tokenizer: Encoding and Decoding

Problem `tokenizer`: Implementing the tokenizer (15 points)

Code deliverable only. No writeup answer.

2.7 Experiments

Problem `tokenizer_experiments`: Experiments with tokenizers (4 points)

(a) Sample 10 documents from TinyStories and 10 from OpenWebText. Using your TinyStories (10K) and OpenWebText (32K) tokenizers, encode these documents into integer IDs. What is each tokenizer's compression ratio (bytes/token)?

Todo

(b) What happens if you tokenize your OpenWebText sample with the TinyStories tokenizer? Compare the compression ratio and/or describe qualitatively what happens.

Todo

(c) Estimate the throughput of your tokenizer (e.g., in bytes/second). How long would it take to tokenize the Pile dataset (825GB of text)?

Todo

(d) Encode the TinyStories and OpenWebText training and development datasets into integer token IDs. Why is `uint16` an appropriate datatype for serializing them?

Todo

3 Transformer Language Model Architecture

3.3 Basic Building Blocks: Linear and Embedding Modules

Problem `linear`: Implementing the linear module (1 point)

Code deliverable only. No writeup answer.

Problem `embedding`: Implement the embedding module (1 point)

Code deliverable only. No writeup answer.

3.4 Pre-Norm Transformer Block

Problem `rmsnorm`: Root Mean Square Layer Normalization (1 point)

Code deliverable only. No writeup answer.

Problem `positionwise_feedforward`: Implement the position-wise feed-forward network (2 points)

Code deliverable only. No writeup answer.

Problem `rope`: Implement RoPE (2 points)

Code deliverable only. No writeup answer.

Problem `softmax`: Implement softmax (1 point)

Code deliverable only. No writeup answer.

Problem `scaled_dot_product_attention`: Implement scaled dot-product attention (5 points)

Code deliverable only. No writeup answer.

Problem `multihead_self_attention`: Implement causal multi-head self-attention (5 points)

Code deliverable only. No writeup answer.

Problem `transformer_block`: Implement the Transformer block (3 points)

Code deliverable only. No writeup answer.

Problem `transformer_lm`: Implementing the Transformer LM (3 points)

Code deliverable only. No writeup answer.

3.6 Resource Accounting

Problem `transformer_accounting`: Transformer LM resource accounting (5 points)

(a) Consider GPT-2 XL with `vocab_size` = 50,257, `context_length` = 1,024, `num_layers` = 48, `d_model` = 1,600, `num_heads` = 25, and `d_ff` = 4,288. How many trainable parameters does the model have? If each parameter is single-precision float, how much memory is needed to load the model?

Todo

(b) Identify the matrix multiplies needed for one forward pass of GPT-2 XL with a `context_length`-token input. List them with descriptions and give the total FLOPs.

Todo

(c) Based on your analysis, which parts of the model require the most FLOPs?

Todo

(d) Repeat your analysis for GPT-2 small (12 layers, 768 `d_model`, 12 heads), GPT-2 medium (24 layers, 1024 `d_model`, 16 heads), and GPT-2 large (36 layers, 1280 `d_model`, 20 heads). Give the FLOP breakdown for each model as a proportion of total forward-pass FLOPs. As the model size grows, which parts take up proportionally more or less of the total?

Todo

(e) Take GPT-2 XL and increase the context length to 16,384. How does the total FLOPs for one forward pass change? How does the relative FLOPs contribution of each component change?

Todo

4 Training a Transformer LM

4.1 Cross-entropy loss

Problem `cross_entropy`: Implement cross-entropy (1 point)

Code deliverable only. No writeup answer.

4.2 The SGD Optimizer

Problem `learning_rate_tuning`: Tuning the learning rate (1 point)

Run the SGD example with learning rates 1e1, 1e2, and 1e3 for 10 training iterations. What happens to the loss for each learning rate? Does it decay faster, decay slower, or diverge?

Todo

4.3 AdamW

Problem `adamw`: Implement AdamW (2 points)

Code deliverable only. No writeup answer.

Problem `adamw_accounting`: Resource accounting for training with AdamW (2 points)

Assume float32 for every tensor.

(a) How much peak memory does running AdamW require? Break it down into parameters, activations, gradients, and optimizer state, in terms of `batch_size`, `vocab_size`, `context_length`, `num_layers`, `d_model`, and `num_heads`. Assume $d_{\text{ff}} = \frac{8}{3} d_{\text{model}}$. Count only the activation components listed in the handout (RMSNorms, attention sublayer, SwiGLU, final RMSNorm, output embedding, and cross-entropy on logits).

Todo

(b) Instantiate your answer for GPT-2 XL to get an expression that depends only on `batch_size`. What is the maximum batch size that fits in 80GB of memory? Give an expression of the form $a \cdot \texttt{batch_size} + b$ and the numerical maximum batch size.

Todo

(c) How many FLOPs does one step of AdamW take? Give an algebraic expression with a brief justification.

Todo

(d) An NVIDIA H100 has a theoretical peak of 495 teraFLOP/s for float32 operations. Assume 50% MFU. Following Kaplan et al. and Hoffmann et al., assume the backward pass has twice

the FLOPs of the forward pass. How many hours would it take to train GPT-2 XL for 400K steps at batch size 1024 on a single H100? Give a brief justification.

Todo

4.4 Learning Rate Scheduling

Problem `learning_rate_schedule`: Implement cosine learning rate schedule with warmup (1 point)

Code deliverable only. No writeup answer.

4.5 Gradient Clipping

Problem `gradient_clipping`: Implement gradient clipping (1 point)

Code deliverable only. No writeup answer.

5 Training Loop

5.1 Data Loader

Problem `data_loading`: Implement data loading (2 points)

Code deliverable only. No writeup answer.

5.2 Checkpointing

Problem `checkpointing`: Implement model checkpointing (1 point)

Code deliverable only. No writeup answer.

5.3 Training Loop

Problem `training_together`: Put it together (4 points)

Code deliverable only. No writeup answer.

6 Generating Text

Problem `decoding`: Decoding (3 points)

Code deliverable only. No writeup answer.

7 Experiments

7.1 How to Run Experiments and Deliverables

Problem `experiment_log`: Experiment logging (3 points)

Submit your logging infrastructure code (code deliverable) and an experiment log. The log should be a document of all the things you tried for the assignment problems in this section.

Todo

7.2 TinyStories

Problem `learning_rate`: Tune the learning rate, 2 B200 hrs (3 points)

(a) Perform a hyperparameter sweep over learning rates and report the final losses (or note divergence). Show learning curves for multiple learning rates, and explain your search strategy. Also report the final model's per-token validation loss on TinyStories, which must be at most 1.45.

Todo

(b) Investigate how the point at which learning rates diverge is related to your best learning rate. Include learning curves of increasing learning rate, with at least one divergent run, and analyze how this relates to convergence rates.

Todo

Problem `batch_size_experiment`: Batch size variations, 1 B200 hr (1 point)

Vary your batch size from 1 to the GPU memory limit. Try several batch sizes, including 64 and 128. Re-optimize the learning rate if necessary.

(a) Learning curves for runs with different batch sizes.

Todo

(b) A few sentences on your findings about batch sizes and their impact on training.

Todo

Problem `generate`: Generate text (1 point)

Using your decoder and trained checkpoint, generate text. You may need to tune temperature, top-p, etc. to get fluent output.

(a) A text dump of at least 256 tokens (or until the first `<|endoftext|>` token).

Todo

(b) A brief comment on the fluency of this output and at least two factors that affect how good or bad it is.

Todo

7.3 Ablations and Architecture Modification

Problem `layer_norm_ablation`: Remove RMSNorm and train, 0.5 B200 hrs (1 point)

Remove all RMSNorms from your Transformer and train. What happens at the previous optimal learning rate? Can you get stability with a lower learning rate?

(a) A learning curve for the run without RMSNorms, and a learning curve for the best learning rate.

Todo

(b) A few sentences on the impact of RMSNorm.

Todo

Problem `pre_norm_ablation`: Implement post-norm and train, 0.5 B200 hrs (1 point)

Modify your pre-norm Transformer into a post-norm one and train it.

(a) A learning curve for the post-norm Transformer, compared to the pre-norm one.

Todo

Problem `no_pos_emb`: Implement NoPE, 0.5 B200 hrs (1 point)

Modify your RoPE Transformer to remove the position embedding information entirely.

(a) A learning curve comparing the performance of RoPE and NoPE.

Todo

Problem `swiglu_ablation`: SwiGLU vs. SiLU, 0.5 B200 hrs (1 point)

Replace SwiGLU with a SiLU feed-forward network, $\mathrm{FFN}_{\mathrm{SiLU}}(x) = W_2\,\mathrm{SiLU}(W_1 x)$, using $d_{\mathrm{ff}} = 4 \times d_{\mathrm{model}}$ so that the parameter count roughly matches.

(a) A learning curve comparing SwiGLU and SiLU feed-forward networks, with approximately matched parameter counts.

Todo

(b) A few sentences discussing your findings.

Todo

7.4 Running on OpenWebText

Problem `main_experiment`: Experiment on OWT, 2 B200 hrs (2 points)

Train your language model on OpenWebText with the same architecture and total training iterations as TinyStories. You may need to re-tune hyperparameters.

(a) A learning curve on OpenWebText. Describe the difference in losses from TinyStories, and how we should interpret these losses.

Todo

(b) Generated text from the OpenWebText model, in the same format as the TinyStories outputs. How fluent is it? Why is the output quality worse even with the same model and compute budget as TinyStories?

Todo

7.5 Your Own Modification + Leaderboard

Problem `leaderboard`: Leaderboard, 10 B200 hrs (6 points)

Train a model under the leaderboard rules (at most 45 minutes on a B200, OpenWebText training data only) to minimize validation loss within 0.75 B200-hours.

(a) The final validation loss, a learning curve with a wall-clock-time x-axis under 45 minutes, and a description of what you did.

Todo